UNIT V VIDEO ANALYTICS

Video Processing – use cases of video analytics-Vanishing Gradient and exploding gradient problem - ResNet architecture-ResNet and skip connections-Inception NetworkGoogleNet architecture Improvement in Inception v2-Video analytics-ResNet and Inception v3.

Video Analytics Using Deep Learning

Videos are a really powerful medium. Roughly 500 hours of videos are uploaded to YouTube every minute. And the number of videos created is increasing daily. With the advent of smartphones and improved hardware, the video quality is enhanced. More videos are being created and stored across domains and geographies. We have movies, advertisements, short clips, and personal videos. Most of the videos contain human faces, objects, and some movements of objects. A video might be shot during the daytime or in the night, under different lighting conditions. We have cameras capturing the movement of pedestrians on the road, manufacturing cameras for monitoring goods and products on manufacturing lines, security cameras for surveillance on airports, and number plate detection and reading systems in the parking lots, to name a few of the video analytics solutions.

Video is a sequence of images. ResNet and Inception network architectures are advanced networks and most sought after for a lot of cutting-edge Deep Learning solutions.

5.1. Video processing

Videos are not new to us. We record videos using our phone, laptops, hand cameras, and so on. YouTube is one of the biggest sources of videos. Advertisements, movies, sports, social media uploads, TikTok videos, and so on are getting created every second. And by analyzing them, we can uncover a lot of insights about the behavior, interactions, timings, and sequence of things. A very powerful medium indeed!

There can be multiple approaches to design a video analytics solution. We can consider video as a collection of frames and then perform analytics by treating the frames as individual images. Or we can also add an additional dimension of sound to it.

Use cases of video analytics

Videos are a rich source of knowledge and information. We can utilize Deep Learning– based capabilities across domains and business functions. Some of them are listed as follows:

1. Real-time face detection can be done using video analytics, allowing us to detect and recognize the faces. It has huge benefits and applications across multiple domains. We have discussed the application in detail in the last chapter.
2. In disaster management, video analytics can play a significant role. Consider

action which directly leads to more lives saved.

3.For crowd management, video analytics plays an important role. We can identify the concentration of the population and the eminent dangers in that situation. The respective team can analyze the videos or a real-time streaming of video using cameras. And a suitable action can be taken to prevent any mishappening.

4. By analyzing the social media videos, the marketing teams can improve the contents. The marketing teams can even analyze the contents of the competitors and tweak their business plan accordingly as per the business needs.

5. For object detection and object tracking, video analytics can quickly come up with a decision if an object is present in the video or not. This can save manual efforts. For example, if we have a collection of videos of different cars and we wish to classify them in different brands, the manual process will be to open each and every video and then take a

decision – which is both time-consuming and error-prone. Using Deep Learning–based video classification, this entire process can be automated.

6. Video analytics can help in the inspection and quality assurance. Instead of manual inspection of each part present in a machine, a video can be taken for the entire process. And then using Deep Learning, the quality inspection can be conducted. These are not the only use cases. There are a number of applications across domains and sectors. With Deep Learning–based solutions, video analytics is really making an impact into the business world.

5.2. Vanishing gradient and exploding gradient problem

The Neural Networks are trained using backpropagation and gradient-based learning methods. During training, we want to reach the most optimum value of weights resulting in minimum loss. Now, each of the weights constantly gets updated during the training of the algorithm. The update is proportional to the partial derivative of the error function with respect to the current weight in each training iteration.

In Figure 7-1, we are showing that in the sigmoid function(activation function), we can face the problem of vanishing gradient, while in the case of a ReLU (Rectified Linear Unit) or Leaky ReLU, we will not have vanishinggradient as an issue.

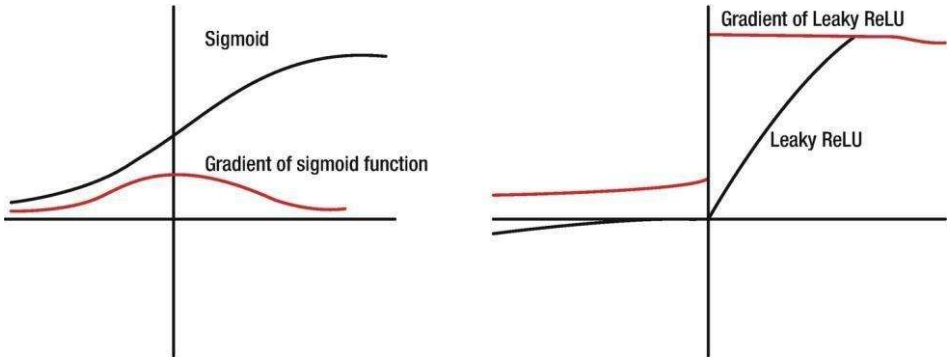


Figure 7-1 Vanishing gradient is a challenge we face with deep Neural Networks. The figure on the left shows that for the sigmoid activation function, we do face a big problem which gets sorted for Leaky ReLU

The challenge can be sometimes this update becomes too small, and hence the weight does not get updated. It results in very less or practically no training of the network. This is referred to as the vanishing gradient problem.

Let's understand the problem in depth now. We are again looking at the basic network architecture in Figure 7-2.

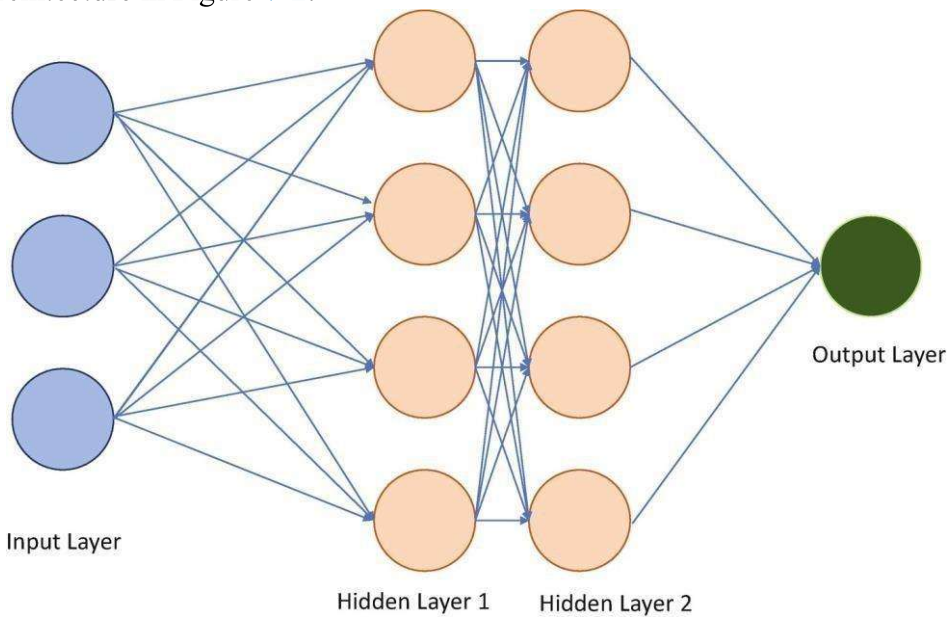


Figure 7-2 Basic neural architecture having an input layer, hidden layers, and an output layer

We know that each of the neurons in the network has an activation function and a bias term. It accepts a finite number of input weight products, adds a bias term to it, and then the activation function is applied on it. The output is then passed to the next neuron.

We also know that in a network, the difference between the expected output and the predicted value is calculated which is nothing but the error term. We would want the error term to be minimized. The error will be minimized when we have achieved the best combination of weights and biases across the layers and neurons which minimizes the error.

When the error is calculated, a gradient descent is applied on the graph of the error function. This gradient descent is the differentiation of the error function with respect to each of the independent variables (weights and biases) present in it. This is the job of the backpropagation algorithm – which takes care of manipulating these weights and biases by a constant term called the learning rate. This is done from the last layer to the first layer in the backward direction or from the right to the left. In each successive iteration, gradient descent is calculated and the direction of change is determined. And hence, the weights and biases are updated till the network minimizes the error – or, in other words, till the error reaches a *global minima* as shown in Figure 7-3. The error gradient hence is the direction and magnitude calculated during the training of the network. It is used to update the weights in the right direction and right magnitude.

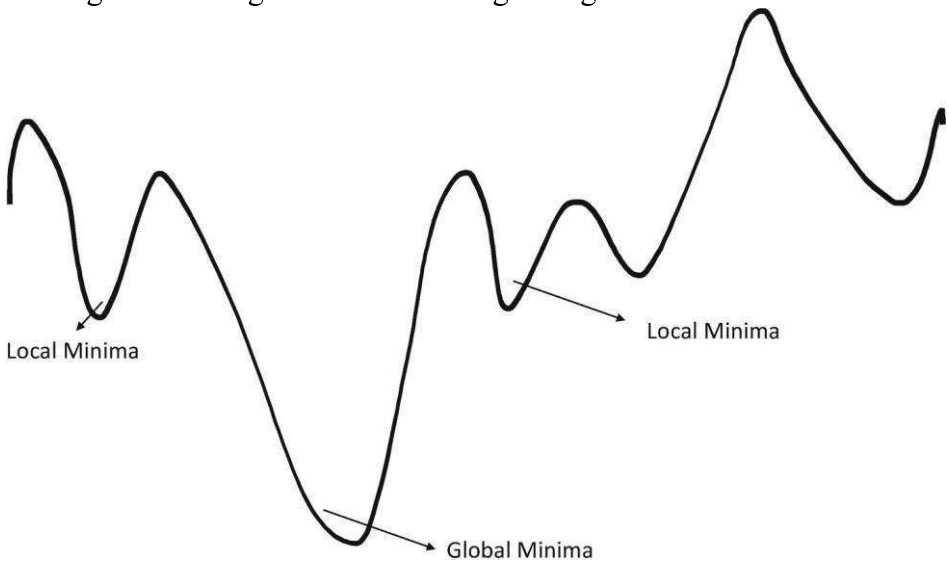


Fig Representing the Local Minima and Global Minima

Now a situation arises wherein if we have a very deep network, the initial layers have a very less impact on the final output as compared to the final layers of the network. Or in other words, the initial layers undergo very less training, and their values undergo very less amount of change. This is due to the fact that the backpropagation computes the gradients using a chain rule from the final layers to the initial layers.

Hence, in an n-layered network, the gradient decreases exponentially with the value of n, and hence the initial layers will train very slowly. Or, in the worst-case scenario, they will stop to train.

There can be multiple signs to check for the vanishing gradient problem:

1. The easiest way to detect vanishing gradient is through kernel weight distribution. If the weights are dying to zero or very very close to zero, we might be encountering a vanishing gradient problem.
2. The model's weight close to the final layers will have more change as compared to the initial layers.

3. The model will not improve or will improve very slowly during the training phase.
4. Sometimes, the training stops early. It means that any further training does not improve the model.

(There are a few suggested solutions for the vanishing
Gradient problem:

1. Generally, reducing the number of layers in the network might help in resolving gradient problems. But at the same time, if the number of layers is reduced, the network's complexity goes down, and it can also impact the performance of the network.
2. The ReLU activation function resolves the vanishing gradient problem. ReLU suffers less from vanishing gradients as compared to tanh or sigmoid activation functions.
3. Residual networks or ResNets are also one of the solutions for this problem. They do not resolve the problem by saving the gradient flow; instead, they usea combination
   or ensemble of multiple smaller networks. And hence, ResNets despite being deep networks are able to achieve lesser loss as compared to shallow networks.

On one hand, we have a vanishing gradient problem, while on the other hand, we have an exploding gradient problem.

In deep networks, error gradients sometimes become very large as they get accumulated. Hence, the updates in the networks will be very large which make the network unstable. There are a few signs of exploding gradients which can help us in detecting exploding gradient:

1. The model is suffering from poor loss during the training phase.
2. During the training of the algorithm, we might encounter NaN for the loss or for the weights.
3. The model is generally unstable, or in other words the updates to loss in subsequent iterations are huge indicating an unstable state.
4. The error gradients are constantly above 1 for each of the layers and neurons in the network.

Exploding gradient can be resolved using

1. We can reduce the number of layers in the network or can try reducing the batch size during training.

2. L1 and L2 weight regularization can be added which will act as a penalty to the network loss functions.

3. *Gradient clipping* is one of the methods which can be used. We can limit the size of the gradients during the process of training. We set a threshold for the error gradients, and the error gradients are set to that limit or *clipped* if the error gradient exceeds the threshold.

4. We can use LSTM (long short-term memory) if we are working with Recurrent Neural Networks. This concept is beyond the scope of this book.

Both vanishing and exploding gradients are a nuisance which will impact the performance of the network. They can make the network unstable and require correction using a few of the options mentioned earlier.

5.3. ResNet architecture

Lot of architectures are used them for image classification, object detection, face recognition, and so on. They are deep Neural Networks and generating good results for us. But in very deep networks, we encounter a problem of vanishing gradients. Residual networks or *ResNets* solve this problem by using skip connections.

Skip connections take the activation from one layer to a much deeper layer in the network which allows us to train even more deep networks, which may be beyond 100 layers. Now, we will discuss ResNet and skip connection in detail in the next section.

5.4.1. ResNet and skip connection

When we talk about Neural Networks and the fantastic performance shown by them, immediately it is attributed to the *depth* of the network. It is assumed that the deeper the network is, the better is the accuracy. The initial layers will learn the basic features, and deeper layers will learn more advanced features.

But it was found, by adding a greater number of layers, we are increasing the complexity of the network. In fact for a deeper network (like 56 layers deep), the loss was greater than a network with less (20) layers.

Note Generally, models using convolutional and fully connected layers between 16 and 30 give the best results for CNN.

This loss can be attributed to the problem of vanishing gradients. To resolve the problem of vanishing gradient, residual blocks are introduced as shown in Fig Residual blocks implement *skip connection* or identity mapping.

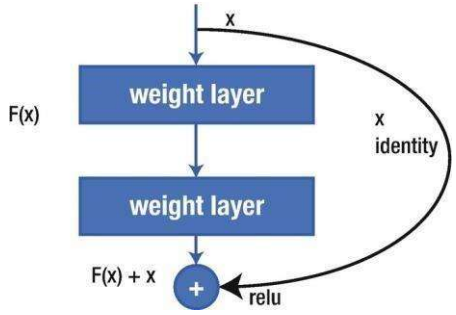
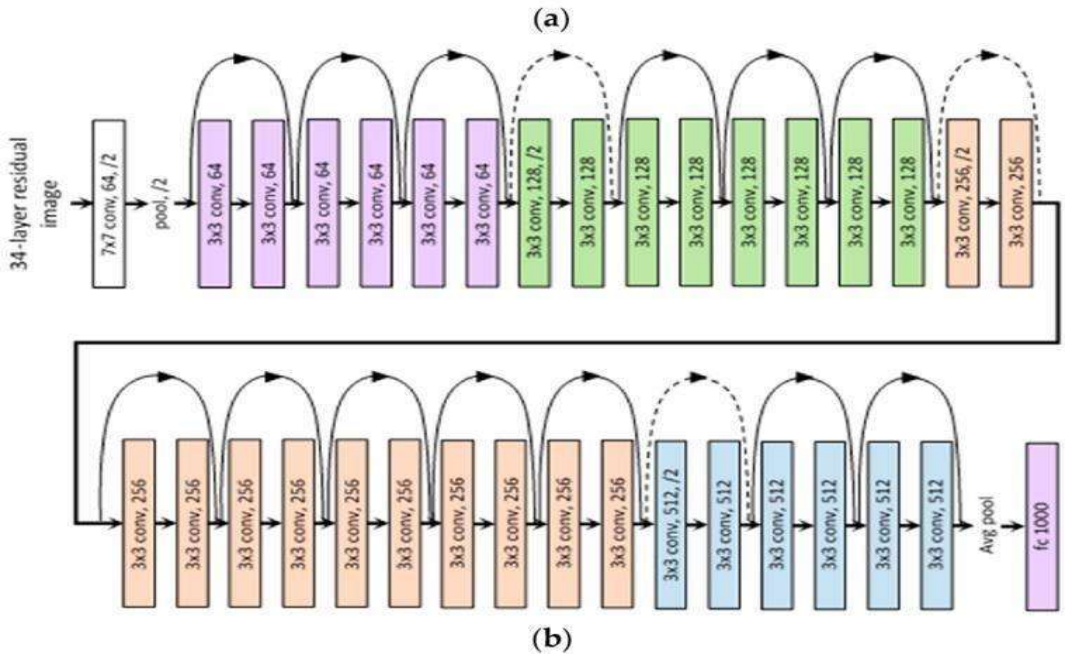


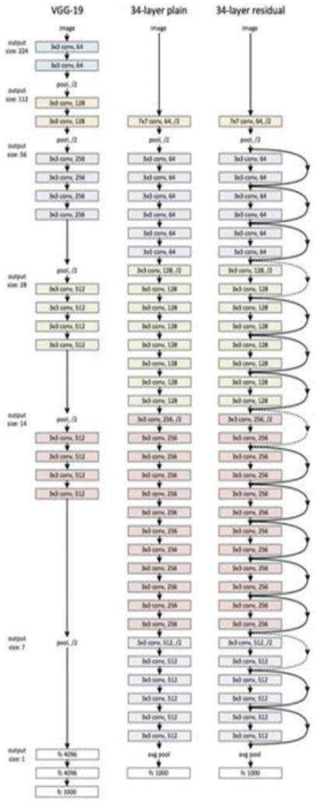
Figure 7-4 Skip connection is the heart of residual networks or ResNets. Note how the output from the previous layer is passed to the next layer, thereby skipping

a layer in between. It allows training deeper networks without the problem of vanishing gradients

The intuition behind a network with residual blocks is that each layer is fed to the next layer of the network and also directly to the next layers skipping between a few layers in between. Residual blocks allow you to train much deeper neural networks. The connection(gray arrow) is called **skip connection or shortcut connection** as it is bypassing one or more layers in between. It is also called **identity connection** as we can learn an identity function from it.This identity mapping has no input parameters of itself; rather, it takes the output from the previous layer and adds to the next layer. In other words, it acts as a shortcut connection before the second activation. Because of this shortcut, it is possible to train even deeper networks without diluting the performance of the network. This is the heart of the solution and the reason for its resounding success.

We are now examining the ResNet-34 architecture





- This is the architecture of ResNet 34
- No fully connected layers and no dropouts were used
- All the convolutions are 3x3
- The dotted lines represent where the dimensions are different.
- To solve the difference in dimensions, the input is down-sample by 2 and then zero padding to match the two dimensions.

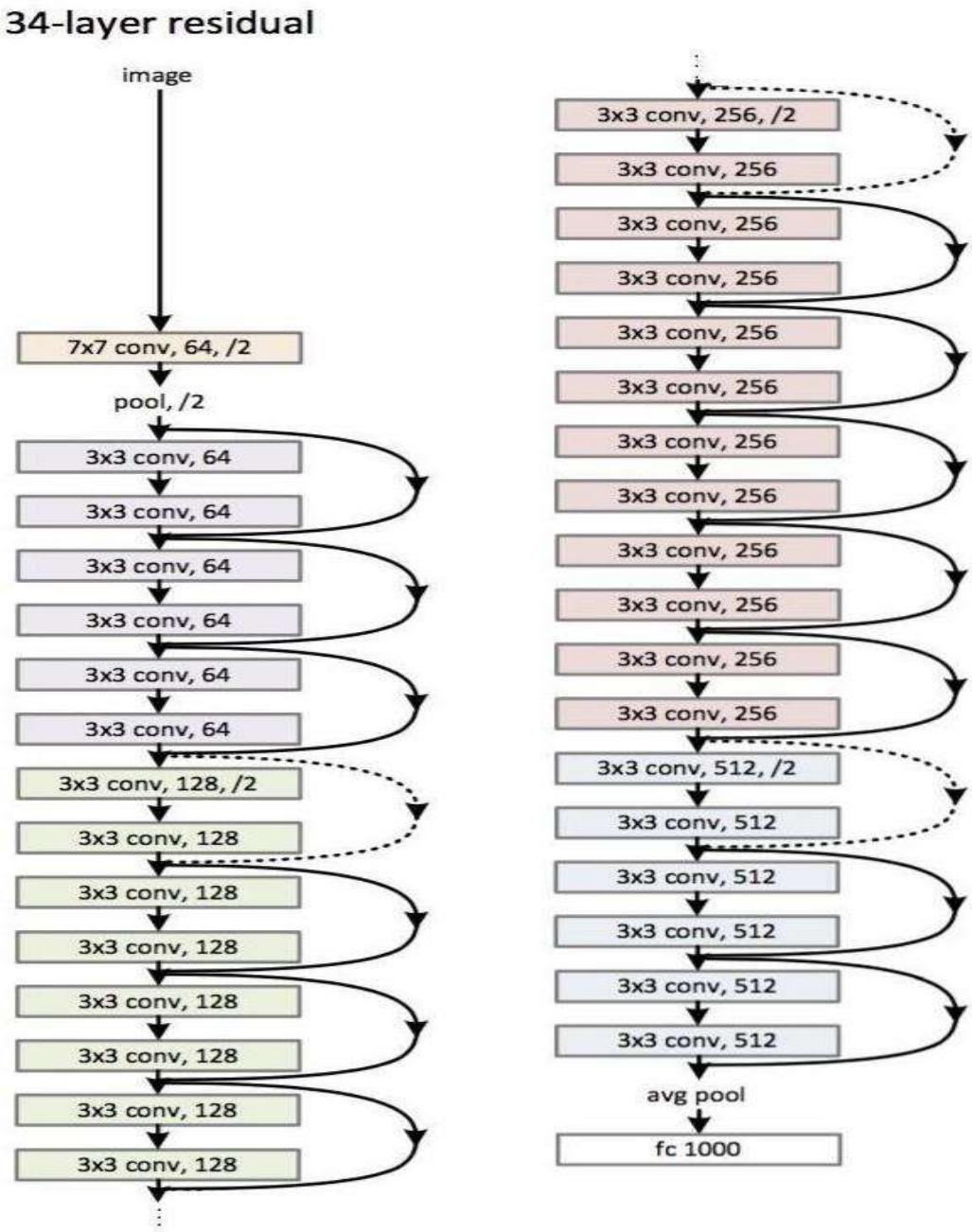


Figure 7-5 **ResNet-34 complete architecture**

Let's go a bit deeper into the network. Observe the four residual blocks in the architecture as shown in Figure

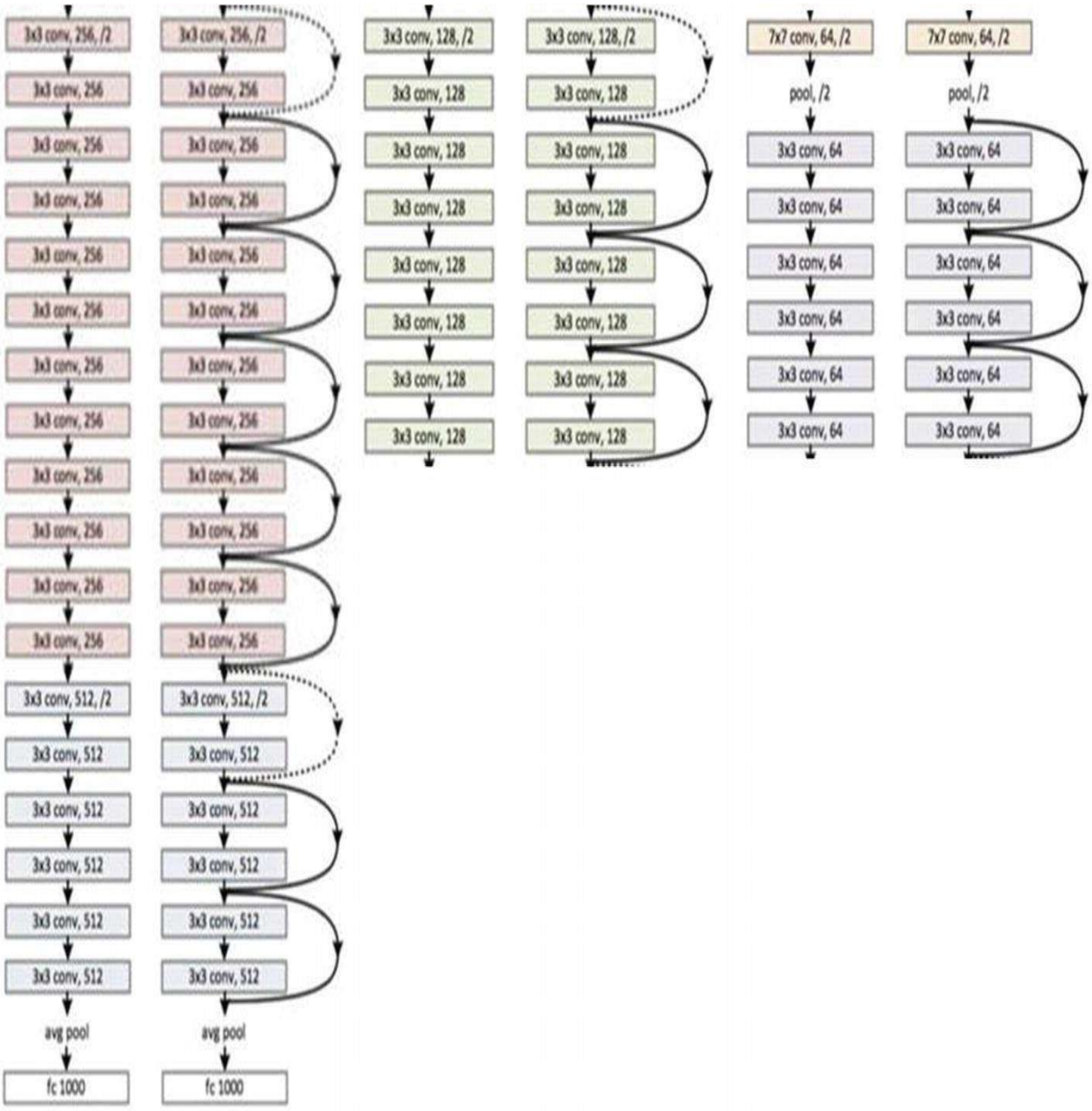


Figure 7-6 Four residual blocks are shown in the figure. Note how for each of the plain networks on the left, we have a corresponding block using skip

connection. Skip connections are allowed to train deeper networks without an adverse impact on the accuracy of the network. For example, for the very first block, we have a plain network having a 7x7 conv layer followed by a 3x3 conv layer. Note the corresponding block using the skip connection

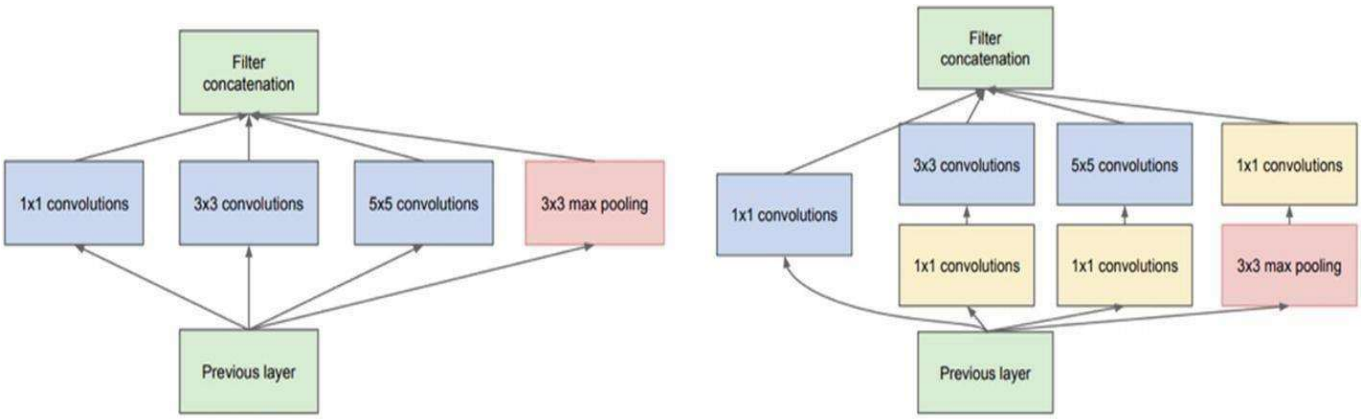
We can analyze that in each residual architecture, skip connection takes the output from the previous layer and shares it two blocks away. This is the core difference with the plain architecture on the left, which boosts the performance for ResNet.Skip connections extend the capabilities of deep networks in a very interesting manner. The inventors tested the network with 100 and 1000 layers on the CIFAR dataset. The inventors found that using an ensemble of residual networks was able to achieve a 3.57% error rate on ImageNet and hence secured first place in the ILSVRC2015 competition.

There are other variants of ResNet gaining popularity like ResNetXt, DenseNet, and so on. These variants explore the changes which can be made to the original ResNet architecture. For example, ResNetXt introduced *cardinality* as one of hyperparameters for the model. We have listed research papers at the end of the chapter for interested audiences.

5.4.2. Inception network

Deep Learning is fantastic when it comes to complex tasks. And we have observed that using stacked convolutional layers, we are able to train deep networks. But there are a few challenges with it:

1. Networks become overcomplicated and demand huge computation power.
2. Vanishing and exploding gradient problems are encountered while training the network.
3. Many times, while observing the training and test accuracy, networks overfit and hence are not useful for unseen datasets.
4. Moreover, choosing the best kernel size is a tough decision. A poorly chosen kernel size will lead to ill- fitting results.



(a) Inception module, naïve version (b) Inception module with dimension reductions

Figure 7-7 On the left, we have the naïve version of the Inception module. In the naïve version, we have 1x1, 3x3, and 5x5 convolutions.

In the first version, a naïve version of Inception, three different sizes of convolution were done – 1x1, 3x3, and 5x5. Additionally, a max pooling of 3x3 was also proposed. All the respective outputs are then stacked and fed to the next Inception module.

But as the computation cost increases, the researchers added an additional 1x1 convolutional layer for dimensionality reduction. This limits the number of input channels, and 1x1 is less computationally expensive than 3x3 or 5x5. A salient feature is the 1x1 convolution is after the max pooling layer.

Using this second version of dimensionality reduction, a full network was created which is known as *GoogLeNet*. The researchers chose the name as an homage to Yann LeCuns pioneering the LeNet-5 architecture.

Before we go deep into studying the GoogLeNet architecture, it is imperative to discuss the uniqueness of 1x1 convolutions.

In deep networks, the number of feature maps increases with the depth of the network. So, if an input image has three channels and a 5x5 filter has to be applied, then a 5x5 filter will be applied in blocks of 5x5x3.

Moreover, if the input is a block of feature maps from another convolution layer having a depth of 64, then a 5x5 filter will be applied in 5x5x64 blocks. It becomes a computationally challenging task. 1x1 filters help in resolving this challenge.

1x1 convolutions are also called *network-in-network*. It is very simple to understand and implement. It has a single feature or weight of each channel in the input. Similar to any other filter, the output is also a single number. It can be used anywhere in the network, does not require any padding, and generates feature maps with exactly the same width and height as the input.

If the number of channels in the 1x1 convolution is the same as the number of channels in the input image, then invariably the output will also contain the same number of 1x1 filters. And there, 1x1 acts as a nonlinearity function. It is shown in Figure 7-8.

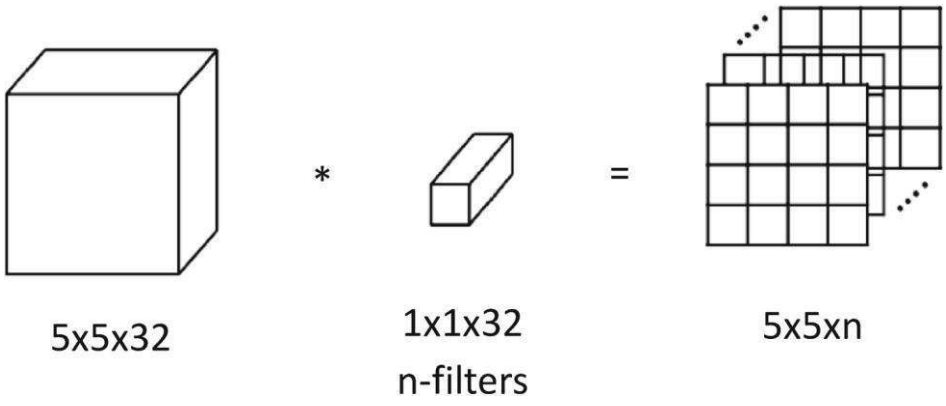


Figure 7-8 1x1 convolutional layer is used to shrink the number of channels. Here, the number of channels in the input and the number of channels in the 1x1 block are the same. Hence, the output has the same number of channels as the number of 1x1 filters

Hence, 1x1 convolution is useful when we want to shrink the number of channels or perform any feature transformation. This results in reducing the computational cost. 1x1 is used in a number of Deep Learning architectures like ResNet and Inception. We will now continue our discussion with the Inception network.

5.4.3. Googlenet Architecture

The complete GoogLeNet architecture is shown in Figure The blocks in blue represent convolution, red are pooling, yellow are softmax, and green are others.

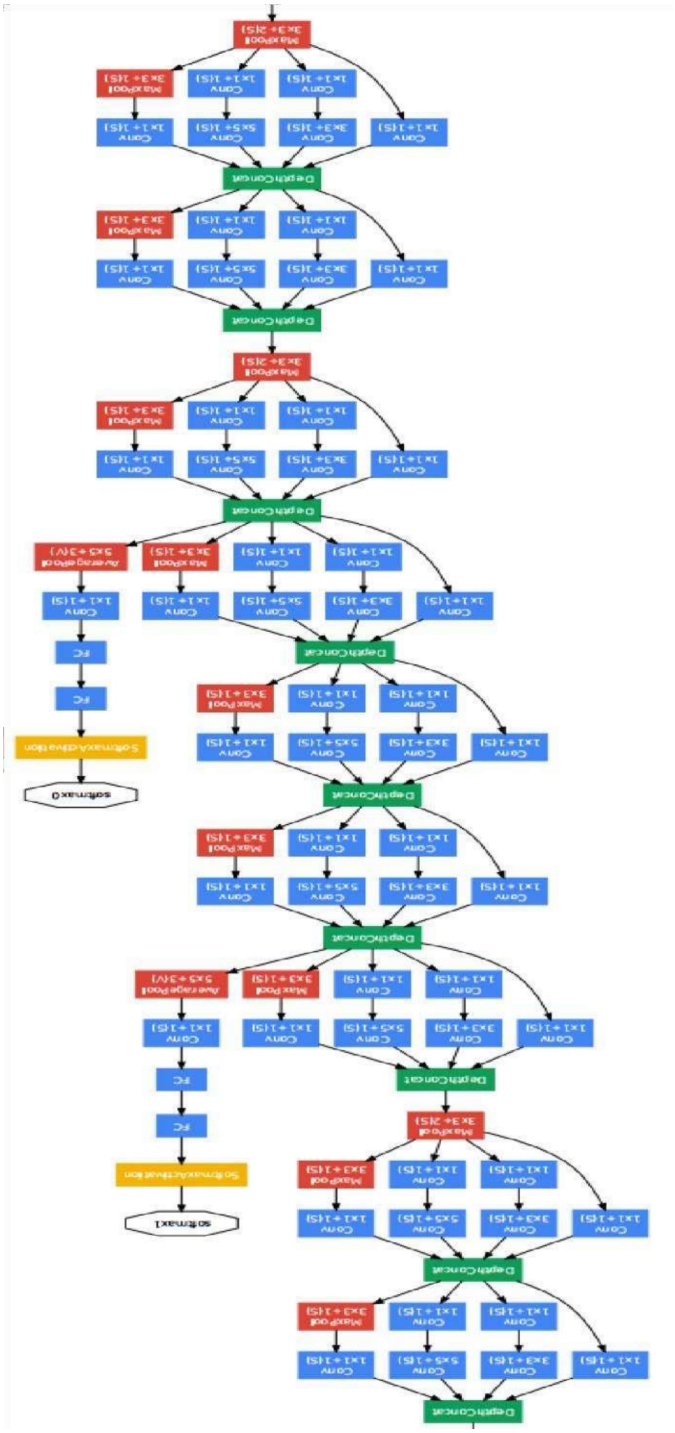


Figure **The complete GoogleNet architecture.**

There are a few important properties about the network:

1. The inception network consists of concatenated blocks of the Inception module.

2. There are nine Inception modules which have been stacked linearly.
3. There are three softmax branches (in yellow in Figure 7-9) at different positions. Out of these three, two are in the middle part of the network acting as auxiliary classifiers. They ensure that the intermediate features are good for the network to learn and give regularization effects.
4. The two softmax compute the auxiliary loss. The net loss is the weighted loss of the auxiliary loss and the real loss. The auxiliary loss is useful during the training and not considered for the final classification.
5. It has 27 layers (22 layers + 5 pooling layers).
6. There are close to 5 million parameters in the network.

We are now zooming in on one of the cropped versions from the network to examine the network better (Figure 7-10). Note how the softmax classifier (shown in the yellow block) has been added – to address the problem of vanishing gradients and overfitting. The final loss is the weighted loss of the auxiliary loss and the real loss of the network.

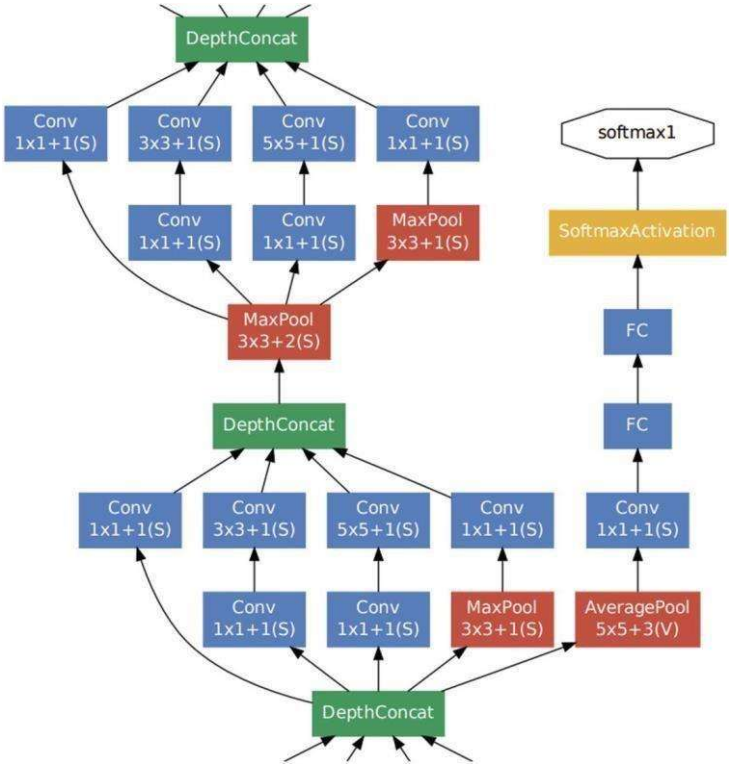


Figure 7-10 A zoomed-in version of a section from the inception network. Note how the softmax classifier has been added (shown in yellow)

Inception v1 proved to be a great solution by getting the first place in

proposing Inception v2 and Inception v3 -is discussed .Improvements in Inception v2 . The motivation was to improve the accuracy and reduce the complexity of the model and hence the computation cost. In Inception v2, there were the following improvements:

1. 5x5 convolutions were factored to two 3x3 convolutions. It was done to improve the computation speed and led to enhanced performance too. It is shown in Figure 7-11. In the figure on the left, we have the original Inception module, and the one on the right is the revised Inception module.

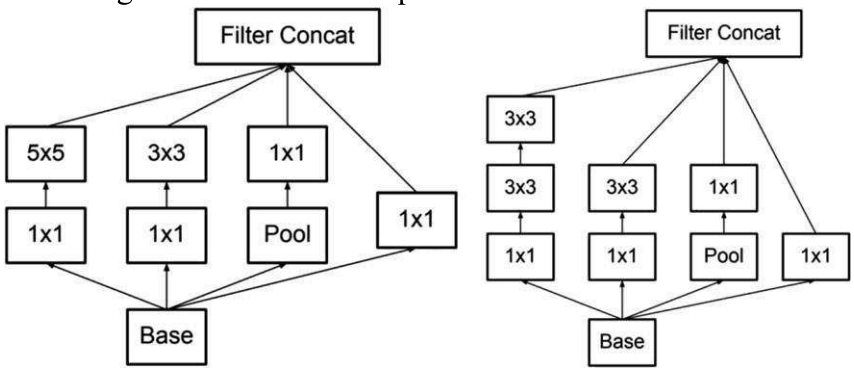


Figure 7-11 Factorization of 5x5 convolutions to two blocks of 3x3 led to the improvement in the computation speed and the overall accuracy of the solution.

.

The second improvement was the convolutions were factorized such that a filter of nxn size is changed to a combination of 1xn and nx1 as shown in Figure 7-12. For example, 5x5 is changed to performing 1x5 first and then 5x1. This further improved the computation efficiency.

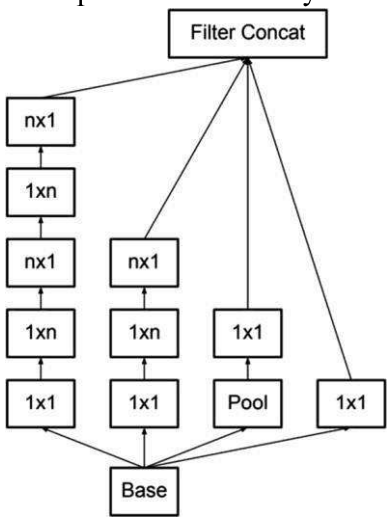


Figure 7-12 **Note how nxn conv can be represented as 1xn and nx1**

With an increase in the depth, the dimensions reduce and hence there can be a loss of information. Hence, an improvement was suggested that the filter banks were made wider instead of going deeper as shown in Figure 7-13.

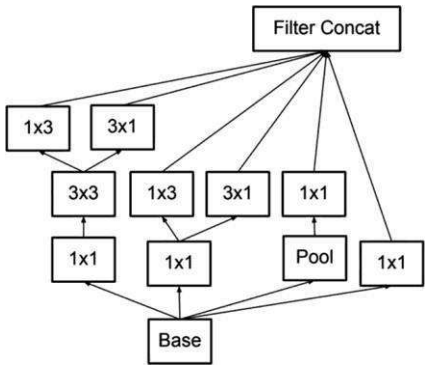


Figure 7-13 The models are made wider instead of deeper. With an increase in depth, the dimensions are reduced drastically.

Although our network is 42 layers deep, our computation cost is only about 2.5 higher than that of GoogLeNet and it is still much more efficient than VGGNet.

Moving ahead, in Inception v3, in addition to the preceding improvements, the significant addition was the use of label smoothing which is a regularizing technique to tackle overfitting. The mathematical proof is beyond the scope of the book. In addition, RMSProp was used as an optimizer, and the auxiliary classifier's fully connected layer is batch normalized. It achieved 3.58% top-5 error on an ensemble of four models which is nearly half of the original

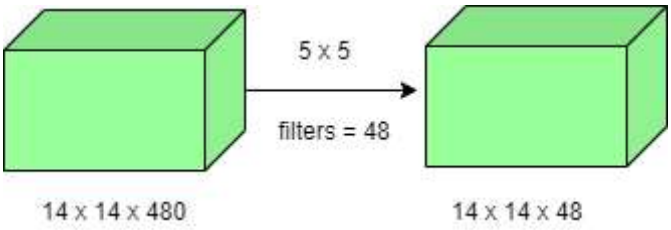
Googlenet architecture :

Google Net (or Inception V1) was proposed by research at Google (with the collaboration of various universities) in 2014 in the research paper titled "Going Deeper with Convolutions". This architecture was the winner at the ILSVRC 2014 image classification challenge. It has provided a significant decrease in error rate as compared to previous winners AlexNet (Winner of ILSVRC 2012) and ZF-Net (Winner of ILSVRC 2013) and significantly less error rate than VGG (2014 runner up). This architecture uses techniques such as 1×1 convolutions in the middle of the architecture and global average pooling.

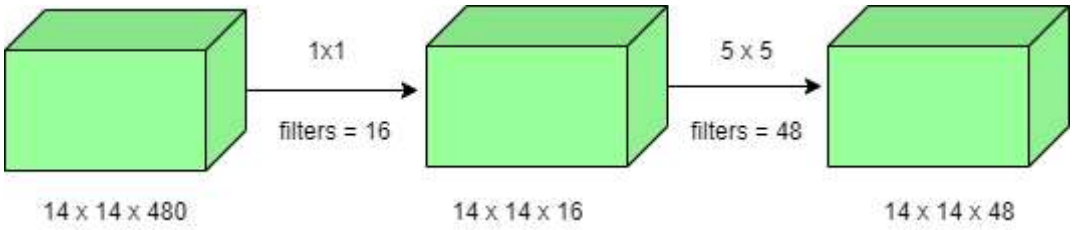
Features of GoogleNet:

The GoogLeNet architecture is very different from previous state-of-the-art architectures such as AlexNet and ZF-Net. It uses many different kinds of methods such as 1×1 convolution and global average pooling that enables it to create deeper architecture. In the architecture, we will discuss some of these methods:

- ***1×1* convolution :** The inception architecture uses 1×1 convolution in its architecture. These convolutions used to decrease the number of parameters (weights and biases) of the architecture. By reducing the parameters we also increase the depth of the architecture. Let's look at an example of a 1×1 convolution below:
    - For Example, If we want to perform 5×5 convolution having 48 filters without using 1×1 convolution as intermediate:



- Total Number of operations : *(14 x 14 x 48) x (5 x 5 x 480) = 112.9 M*
    - With 1×1 convolution :



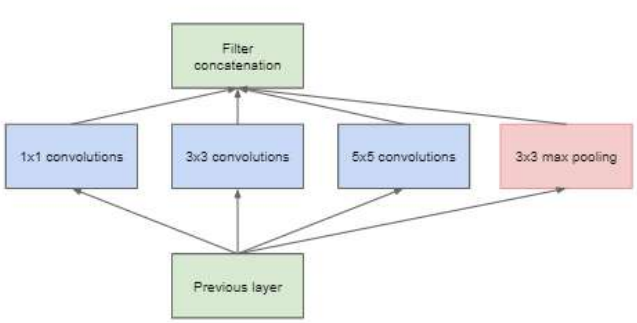
- *(14 x 14 x 16) x (1 x 1 x 480) + (14 x 14 x 48) x (5 x 5 x 16) = 1.5M + 3.8M = 5.3M* which is much smaller than 112.9M.
- **Global Average Pooling :**

  In the previous architecture such as AlexNet, the fully connected layers are used at the end of the network. These fully connected layers contain the majority of parameters of many architectures that causes an increase in computation cost.

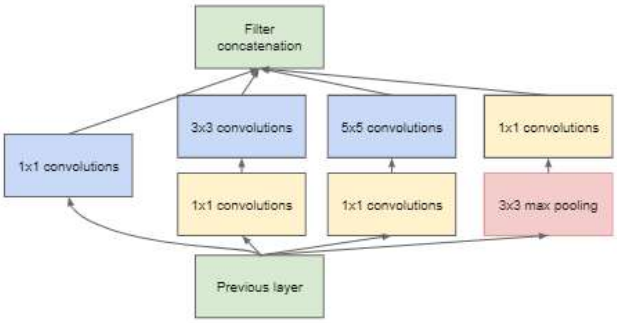
  In GoogLeNet architecture, there is a method called global average pooling is used at the end of the network. This layer takes a feature map of *7×7* and averages it to *1×1*. This also decreases the number of trainable parameters to 0 and improves the top-1 accuracy by 0.6%
- **Inception Module:**

  The inception module is different from previous architectures such as AlexNet, ZF-Net. In this architecture, there is a fixed convolution size for each layer.

  In the Inception module *1×1, 3×3, 5×5* convolution and *3×3* max pooling performed in a parallel way at the input and the output of these are stacked together to generated final output. The idea behind that convolution filters of different sizes will handle objects at multiple scale better.



(a) Inception module, naïve version



(b) Inception module with dimension reductions

- **Auxiliary Classifier for Training:**

  Inception architecture used some intermediate classifier branches in the middle of the architecture.

layer with a stride of 3, a *1×1* convolutions with *128* filters, two fully connected layers of 1024 outputs and 1000 outputs and a softmax classification layer. The generated loss of these layers added to total loss with a weight of 0.3. These layers help in combating gradient vanishing problem and also provide regularization.

Part-A

1. What is the purpose of skip connections and how are they useful?
2. What is the problem of vanishing gradients and how can we rectify it?
3. What is the improvement between Inception v1 and Inception v3 networks?
4. What are the use cases of video analytics?
5. Discuss about video analytics.
6. List the uses of skip connection.
7. Discuss about inception v1.
8. What is inception v2.
9. Discuss about inception v3.
10. What is the use of GoogleNet.
11. What is the use of ResNet.

Part-B

1. Explain about Video Processing.
2. Discuss about various use cases of video analytics
3. Elaborate about Vanishing Gradient and exploding gradient problem
4. Explain about ResNet architecture.
5. Illustrate ResNet and skip connections.

6. Discuss about Inception Network.
7. Elaborate about GoogleNet architecture
8. Explain about Improvement in Inception v2
9. Develop a python code for Implementation of ResNet
10. Develop a python code for Inception v3.